Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Методы машинного обучения

ОТЧЕТ

по лабораторной работе №3

Вариант 7

Выполнил
Студент гр. 351002 Михальков А. Д.

Проверил Лебедевич А. В.

Минск 2026

СОДЕРЖАНИЕ

1 Цель выполнения 3
2 Теоретическая основа 3
2.1 Линейная регрессия и функция потерь 3
2.2 Метод градиентного спуска 3
2.3 Скорость обучения и масштабирование признаков 4
2.4 Стохастический градиентный спуск 4
2.5 Переобучение и регуляризация 5
3 Индивидуальное задание 6
4 Код решения 7
5 Скриншоты выполнения 11
6 Выводы 17

1 Цель выполнения

Изучение метода градиентного спуска как основного алгоритма численной оптимизации в машинном обучении и приобретение практических навыков его самостоятельной реализации для задачи линейной регрессии. Исследование влияния скорости обучения на сходимость алгоритма, освоение стохастического градиентного спуска и сравнение скорости сходимости пакетного и стохастического вариантов по среднеквадратичной ошибке. Изучение L2-регуляризации (Ridge) как способа борьбы с переобучением, а также L1-регуляризации (Lasso), и анализ изменения весов модели в зависимости от коэффициента регуляризации.

2 Теоретическая основа

2.1 Линейная регрессия и функция потерь

В задаче линейной регрессии значение целевой переменной моделируется линейной комбинацией признаков: $\hat{y} = Xw$, где X – матрица признаков размера $l \times n$ (l – число объектов), в которую добавлен столбец единиц для свободного члена w_0, а w – вектор весов. Качество модели оценивается функцией потерь – среднеквадратичной ошибкой (MSE):

$$L(w) = (1/l) \cdot \|Xw - y\|^2 \quad (1)$$

Функция MSE является выпуклой и гладкой, имеет единственный минимум и в каждой точке дифференцируема. Её градиент по вектору весов равен:

$$\nabla L(w) = (2/l) \cdot X^T(Xw - y) \quad (2)$$

Веса, минимизирующие MSE, можно найти аналитически из нормального уравнения $w = (X^TX)^{-1}X^Ty$ (метод наименьших квадратов, МНК). Однако обращение матрицы требует $O(n^3)$ операций и неприменимо при большом числе признаков, поэтому на практике используют итерационные методы – в первую очередь градиентный спуск. Решение МНК служит эталоном для проверки корректности реализации.

2.2 Метод градиентного спуска

Градиент показывает направление наибольшего роста функции, поэтому для уменьшения ошибки веса необходимо изменять в направлении антиградиента. На каждой итерации веса обновляются по формуле:

$$w_{new} = w_{old} - \eta \cdot \nabla L(w_{old}) \quad (3)$$

где η – скорость обучения (learning rate). Итерации продолжаются до сходимости, которая определяется как ситуация, когда вектор весов от шага к шагу меняется незначительно, то есть евклидова (L2) норма его изменения ($\|x\| = \sqrt{(\Sigma x_i^2)}$) не превышает заданного малого числа ε:

$$\|w_{new} - w_{old}\| < \varepsilon \quad (4)$$

Пошаговый алгоритм: 1) инициализировать начальные веса (нулями или значениями из нормального распределения) и число ε; 2) организовать цикл по числу итераций; 3) рассчитать новые веса по формуле (3); 4) завершить цикл, если выполнено условие (4) или достигнуто максимальное число итераций. Так как на каждой итерации используется вся выборка, метод называют пакетным (batch) градиентным спуском.

2.3 Скорость обучения и масштабирование признаков

Скорость обучения η – ключевой гиперпараметр. При слишком малом значении сходимость монотонная, но очень медленная; при умеренном – быстрая и монотонная; при слишком большом шаг «перепрыгивает» минимум, ошибка растёт, и метод расходится. Для MSE условие устойчивости определяется гессианом H, и градиентный спуск сходится при:

$$\eta < 2 / \lambda_{max}(H), \quad H = (2/l) \cdot X^T X \quad (5)$$

Если признаки имеют несопоставимые масштабы, собственные числа гессиана сильно различаются: для одних весов шаг слишком велик, для других – мал, и сходимость замедляется. Масштабирование (стандартизация StandardScaler приводит признаки к нулевому среднему и единичной дисперсии) выравнивает собственные числа, позволяет использовать одну скорость обучения для всех весов и заранее оценить порог устойчивости.

2.4 Стохастический градиентный спуск

В стохастическом градиентном спуске (Stochastic Gradient Descent, SGD) на каждой итерации градиент вычисляется не по всей выборке, а по одному случайно выбранному объекту i:

$$\nabla L_i(w) = 2x_i(x_i^T w - y_i) \quad (6)$$

Стоимость шага снижается с O(l·n) до O(n), поэтому веса обновляются значительно чаще при тех же вычислительных затратах. Платой является шум: ошибка на всей выборке изменяется немонотонно, на отдельных итерациях

может увеличиваться, но в целом убывает. При постоянной η метод не сходится точно в минимум, а колеблется вокруг него с амплитудой, пропорциональной η; критерий (4) для SGD срабатывает редко, поэтому число итераций ограничивают. Промежуточным вариантом является мини-батч градиентный спуск, использующий небольшую случайную подвыборку.

2.5 Переобучение и регуляризация

Признаком переобучения линейной модели служат большие по модулю веса. Регуляризация ограничивает веса, добавляя к функции потерь штраф за сложность модели. L2-регуляризация (Ridge) штрафует сумму квадратов весов; её вклад в градиент равен 2λw:

$$L_{L2}(w) = L(w) + \lambda \cdot \Sigma\, w_j^2, \qquad \nabla L_{L2} = \nabla L + 2\lambda w \tag{7}$$

Здесь λ – коэффициент регуляризации; свободный член w_0 обычно не штрафуется. Чем больше λ, тем сильнее сжимаются веса: при малых λ решение близко к МНК, при больших веса стремятся к нулю (недообучение), модель вырождается в константу. L2 сжимает все веса плавно, но не обнуляет их точно, сохраняя вклад всех признаков.

L1-регуляризация (Lasso) использует сумму модулей весов: $L_{L1}(w) = L(w) + \lambda \cdot \Sigma|w_j|$. Штраф негладок в нуле, поэтому применяют проксимальный градиентный спуск: после обычного шага выполняется «мягкое пороговое» сжатие $w_j \leftarrow \text{sign}(w_j) \cdot \max(|w_j| - \eta\lambda, 0)$. В отличие от L2, L1 точно обнуляет веса наименее значимых признаков и может использоваться для отбора признаков. Подходящее λ выбирают перебором по логарифмической сетке с оценкой качества на отложенной выборке.

3 Индивидуальное задание

В рамках лабораторной работы (вариант 7 – нечётный, реализуется L2-регуляризация) необходимо выполнить следующие этапы:

1) Сгенерировать датасет при помощи sklearn.datasets.make_regression (признаков не менее 2 без учёта целевого, то есть не менее 3 весов), при необходимости выполнить масштабирование данных.
2) Самостоятельно написать функцию градиентного (не стохастического) спуска по алгоритму: инициализировать начальные веса и ε; организовать цикл по количеству итераций; рассчитать новые веса; завершить цикл, если $\|w(k+1) - w(k)\| < \varepsilon$, где w(k) и w(k+1) – веса до и после обновления.
3) Поэкспериментировать со скоростью обучения и сделать выводы.
4) Добавить в функцию выбор случайного объекта на каждой итерации для нахождения весов, чтобы получить функцию стохастического градиентного спуска.
5) Обучить линейную модель при помощи градиентного и стохастического градиентного спуска; нанести среднеквадратичную ошибку обоих методов на один график и сделать выводы о разнице скорости сходимости.
6) Реализовать L2-регуляризацию (нечётный вариант); взять 20 коэффициентов регуляризации в широком диапазоне (от 0,001 до 1000) и показать на графике, как изменяются веса в зависимости от коэффициента регуляризации. Дополнительно реализована L1-регуляризация.

4 Код решения

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_regression
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso

np.random.seed(42)

plt.rcParams['figure.figsize'] = (11, 6)
plt.rcParams['font.size'] = 12
plt.rcParams['axes.grid'] = True
plt.rcParams['grid.alpha'] = 0.3

X, y, true_coef = make_regression(
    n_samples=500,
    n_features=3,
    n_informative=3,
    noise=15,
    bias=5,
    coef=True,
    random_state=42
)

X_scaled = StandardScaler().fit_transform(X)

X_b = np.c_[np.ones(X_scaled.shape[0]), X_scaled]

print('Форма X_b:', X_b.shape, '| Форма y:', y.shape)
print('Истинные коэффициенты (до масштабирования):', np.round(true_coef, 2))
print('Среднее признаков после масштабирования:', X_scaled.mean(axis=0).round(3))
print('Std признаков после масштабирования:    ', X_scaled.std(axis=0).round(3))


def mserror(X, w, y):
    return np.mean((np.dot(X, w).ravel() - y.ravel()) ** 2)


w_exact = np.linalg.solve(X_b.T @ X_b, X_b.T @ y)
mse_exact = mserror(X_b, w_exact, y)

print('\n--- ЭТАЛОН (МНК) ---')
print('Веса по МНК:            ', w_exact.round(4))
print('Минимально достижимая MSE:', round(mse_exact, 4))

def gradient_descent(X, y, eta=0.1, max_iter=10000, eps=1e-8, stochastic=False,
                     w0=None, seed=0):
    rng = np.random.default_rng(seed)
    l, n = X.shape
    w = np.zeros(n) if w0 is None else w0.copy()
    w_hist, errors = [w.copy()], [mserror(X, w, y)]

    for it in range(1, max_iter + 1):
        if stochastic:
            i = rng.integers(l)
            grad = 2 * X[i] * (X[i].dot(w) - y[i])
        else:
            grad = 2 / l * X.T.dot(X.dot(w) - y)

        w_new = w - eta * grad
        dist = np.linalg.norm(w_new - w)
        w = w_new

        w_hist.append(w.copy())
        errors.append(mserror(X, w, y))
```

```
        if dist < eps:
            break
        if not np.isfinite(errors[-1]):
            break
    return w, np.array(w_hist), np.array(errors), it

w_gd, w_hist_gd, err_gd, n_gd = gradient_descent(X_b, y, eta=0.1)
print('Итераций:', n_gd)
print('Веса GD :', w_gd.round(4))
print('Веса МНК:', w_exact.round(4))
print('MSE GD  :', round(err_gd[-1], 4), '| MSE МНК:', round(mse_exact, 4))

H = 2 / len(y) * X_b.T @ X_b
eig = np.linalg.eigvalsh(H)
print('Собственные числа гессиана:', eig.round(3))
print('Критическая скорость обучения 2/lambda_max =', round(2 / eig.max(), 4))

etas = [0.001, 0.01, 0.1, 0.5, 0.9, 1.0]
max_it = 300
fig, ax = plt.subplots(1, 2, figsize=(15, 5.5))

print(f"{'eta':>7} | {'итераций':>8} | {'итоговая MSE':>14} | результат")
for eta in etas:
    with np.errstate(all='ignore'):
        w, wh, err, n = gradient_descent(X_b, y, eta=eta, max_iter=max_it, eps=1e-8)
    diverged = (not np.isfinite(err[-1])) or err[-1] > err[0]
    status = 'расходится' if diverged else ('сошёлся' if n < max_it else 'не успел сойтись за
%d ит.' % max_it)
    print(f'{eta:>7} | {n:>8} | {err[-1]:>14.4f} | {status}')
    ax[0].plot(err[:60], label=f'η={eta}')
    ax[1].plot(err, label=f'η={eta}')

for a, t in zip(ax, ['Первые 60 итераций', f'Все итерации (до {max_it})']):
    a.set_yscale('log'); a.set_xlabel('Итерация'); a.set_ylabel('MSE (лог. шкала)')
    a.set_title(t); a.legend()
ax[0].set_ylim(top=err[0] * 3)
plt.suptitle('Влияние скорости обучения на сходимость GD'); plt.tight_layout(); plt.show()

etas_big = [0.95, 1.0, 1.05, 1.2]
plt.figure(figsize=(9, 5))
for eta in etas_big:
    with np.errstate(all='ignore'):
        _, _, err, n = gradient_descent(X_b, y, eta=eta, max_iter=200, eps=1e-8)
    plt.plot(err, label=f'η={eta}')
plt.yscale('log'); plt.ylim(1e1, 1e12)
plt.xlabel('Итерация'); plt.ylabel('MSE (лог. шкала)')
plt.title('Скорости обучения около критического порога'); plt.legend(); plt.show()

sgd_iters = 5000
etas_sgd = [0.001, 0.005, 0.01, 0.05, 0.1]
plt.figure(figsize=(11, 5.5))
print(f"{'eta':>7} | {'MSE в конце (ср. по посл. 500 ит.)':>36}")
for eta in etas_sgd:
    _, _, err, _ = gradient_descent(X_b, y, eta=eta, max_iter=sgd_iters, stochastic=True,
eps=0)
    print(f'{eta:>7} | {err[-500:].mean():>36.3f}')
    plt.plot(err, label=f'η={eta}', alpha=0.8)
plt.axhline(mse_exact, color='k', ls='--', label='MSE МНК')
plt.yscale('log'); plt.xlabel('Итерация'); plt.ylabel('MSE (лог. шкала)')
plt.title('SGD: влияние скорости обучения'); plt.legend(); plt.show()

n_iter = 600
w_gd, _, err_gd, _   = gradient_descent(X_b, y, eta=0.1,  max_iter=n_iter, eps=0)
w_sgd, _, err_sgd, _ = gradient_descent(X_b, y, eta=0.01, max_iter=n_iter, eps=0,
stochastic=True)

plt.figure(figsize=(11, 6))
plt.plot(err_gd,  label='GD (η=0.1)', lw=2)
plt.plot(err_sgd, label='SGD (η=0.01)', lw=1.5, alpha=0.85)
```

```
plt.axhline(mse_exact, color='k', ls='--', label='MSE МНК (минимум)')
plt.yscale('log'); plt.xlabel('Итерация'); plt.ylabel('MSE на всей выборке (лог. шкала)')
plt.title('Сходимость GD и SGD'); plt.legend(); plt.show()

print('Веса МНК:', w_exact.round(3))
print('Веса GD :', w_gd.round(3),  '| MSE =', round(mserror(X_b, w_gd, y), 3))
print('Веса SGD:', w_sgd.round(3), '| MSE =', round(mserror(X_b, w_sgd, y), 3))

l = len(y)
iters_gd = 60
_, _, err_gd2, _  = gradient_descent(X_b, y, eta=0.1,  max_iter=iters_gd, eps=0)
_, _, err_sgd2, _ = gradient_descent(X_b, y, eta=0.01, max_iter=iters_gd * l, eps=0,
stochastic=True)

plt.figure(figsize=(11, 6))
plt.plot(np.arange(len(err_gd2)) * l, err_gd2, label='GD (η=0.1)', lw=2)
plt.plot(err_sgd2, label='SGD (η=0.01)', lw=1.2, alpha=0.85)
plt.axhline(mse_exact, color='k', ls='--', label='MSE МНК')
plt.yscale('log'); plt.xlabel('Число обработанных объектов'); plt.ylabel('MSE (лог. шкала)')
plt.title('GD и SGD при одинаковых вычислительных затратах'); plt.legend(); plt.show()

def first_below(err, thr):
    idx = np.where(err <= thr)[0]
    return int(idx[0]) if len(idx) else None
thr = mse_exact * 1.05
a, b = first_below(err_gd2, thr), first_below(err_sgd2, thr)
print(f'MSE ≤ {thr:.1f} (в пределах 5% от минимума):')
print('  GD :', None if a is None else f'{a} итераций = {a*l} обработанных объектов')
print('  SGD:', None if b is None else f'{b} итераций = {b} обработанных объектов')

def gradient_descent_l2(X, y, lam, max_iter=50000, eps=1e-10):
    l, n = X.shape
    eta = 0.9 / (1 + lam)
    w = np.zeros(n)
    for it in range(1, max_iter + 1):
        grad = 2 / l * X.T.dot(X.dot(w) - y)
        pen = 2 * lam * w
        pen[0] = 0
        w_new = w - eta * (grad + pen)
        if np.linalg.norm(w_new - w) < eps:
            w = w_new
            break
        w = w_new
    return w, it

lambdas = np.logspace(-3, 3, 20)
W_l2, iters_l2 = [], []
for lam in lambdas:
    w, it = gradient_descent_l2(X_b, y, lam)
    W_l2.append(w); iters_l2.append(it)
W_l2 = np.array(W_l2)

def ridge_closed(X, y, lam):
    l, n = X.shape
    P = np.eye(n); P[0, 0] = 0
    return np.linalg.solve(X.T @ X / l + lam * P, X.T @ y / l)
W_check = np.array([ridge_closed(X_b, y, lam) for lam in lambdas])
print('Макс. отклонение GD от аналитического решения Ridge:', np.abs(W_l2 - W_check).max())
print('Макс. число итераций:', max(iters_l2))

fig, ax = plt.subplots(1, 2, figsize=(15, 5.5))
for j in range(1, X_b.shape[1]):
    ax[0].plot(lambdas, W_l2[:, j], marker='o', ms=4, label=f'w{j}')
ax[0].plot(lambdas, W_l2[:, 0], marker='o', ms=4, ls=':', c='gray', label='w0 (не
штрафуется)')
ax[0].set_xscale('log'); ax[0].set_xlabel('λ (лог. шкала)'); ax[0].set_ylabel('Значение веса')
ax[0].set_title('L2: веса в зависимости от λ'); ax[0].legend()

ax[1].plot(lambdas, np.linalg.norm(W_l2[:, 1:], axis=1), marker='o', c='C3')
```

```
ax[1].set_xscale('log'); ax[1].set_xlabel('λ (лог. шкала)'); ax[1].set_ylabel('||w|| (без
w0)')
ax[1].set_title('L2: норма вектора весов')
plt.tight_layout(); plt.show()

import pandas as pd
pd.set_option('display.float_format', lambda v: f'{v:,.4f}')
df_l2 = pd.DataFrame(W_l2, columns=['w0', 'w1', 'w2', 'w3'], index=pd.Index(lambdas.round(4),
name='λ'))
df_l2['MSE'] = [mserror(X_b, w, y) for w in W_l2]
df_l2
```

5 Скриншоты выполнения

```
Форма X_b: (500, 4) | форма y: (500,)
Истинные коэффициенты (до масштабирования): [26.98 82.5  49.83]
Среднее признаков после масштабирования: [ 0. -0. -0.]
Std признаков после масштабирования:    [1. 1. 1.]

Веса по МНК: [ 8.5837 24.9828 82.8755 51.6372]
Минимально достижимая MSE: 218.8976
```

Рисунок 1 – Параметры датасета и аналитическое решение по МНК

```
Итераций: 100
Веса GD : [ 8.5837 24.9828 82.8755 51.6372]
Веса МНК: [ 8.5837 24.9828 82.8755 51.6372]
MSE GD  : 218.8976 | MSE МНК: 218.8976
```

Рисунок 2 – Результат работы градиентного спуска (η = 0,1)

```
Собственные числа гессиана: [1.891 2.    2.021 2.089]
Критическая скорость обучения 2/lambda_max = 0.9576
```

Рисунок 3 – Собственные числа гессиана и критическая скорость обучения

```
   eta | итераций |   итоговая MSE | результат
 0.001 |      300 |      3285.6415 | не успел сойтись за 300 ит.
  0.01 |      300 |       218.9492 | не успел сойтись за 300 ит.
   0.1 |      100 |       218.8976 | сошёлся
   0.5 |        9 |       218.8976 | сошёлся
   0.9 |      183 |       218.8976 | сошёлся
   1.0 |      300 | 66487468482898540361678848.0000 | расходится
```

Рисунок 4 – Результаты экспериментов со скоростью обучения для GD

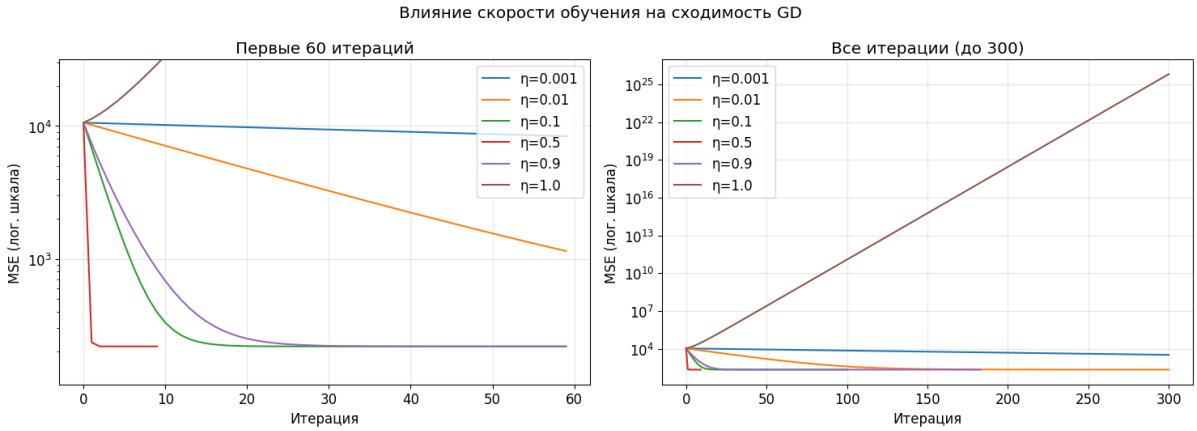


Рисунок 5 – Влияние скорости обучения на сходимость градиентного спуска

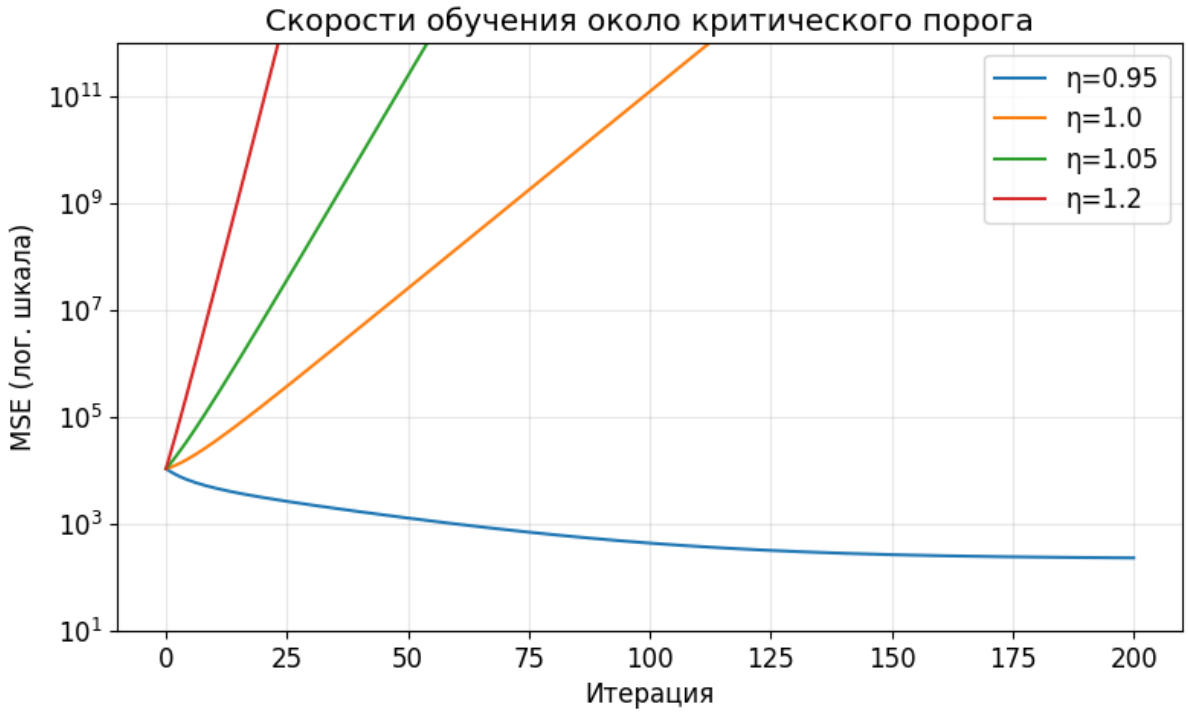


Рисунок 6 – Поведение GD при скоростях обучения около критического порога

eta	MSE в конце (ср. по посл. 500 ит.)
0.001	219.391
0.005	223.732
0.01	229.351
0.05	282.496
0.1	412.053

Рисунок 7 – Средняя MSE стохастического спуска при разных скоростях обучения

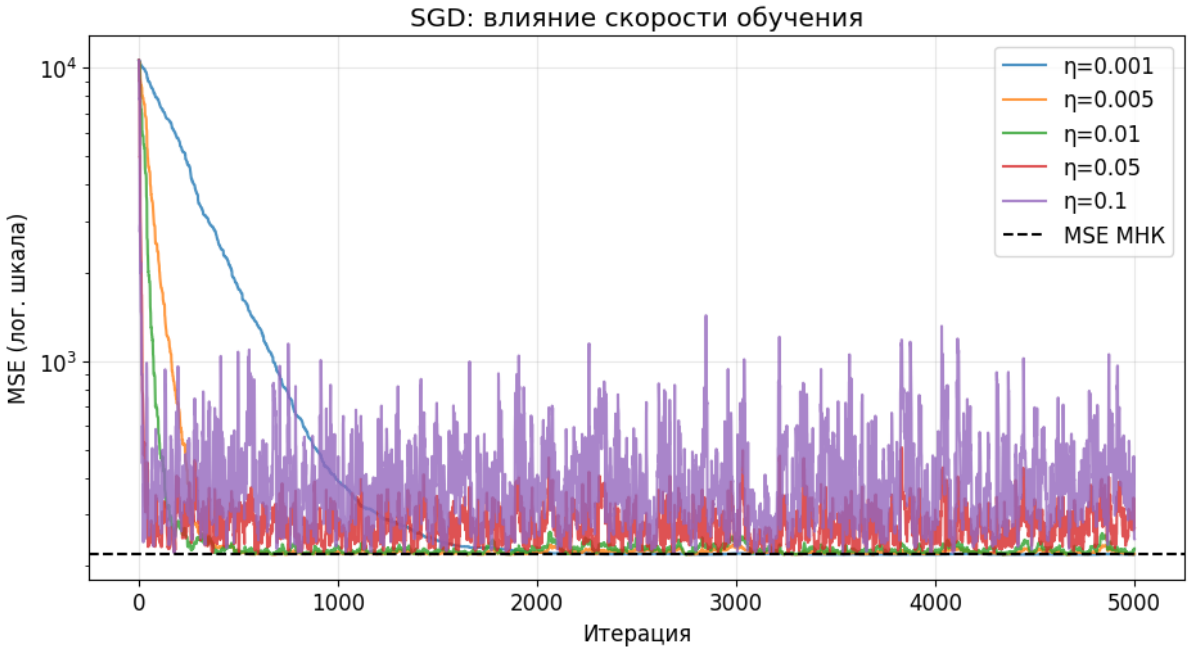


Рисунок 8 – Стохастический градиентный спуск: влияние скорости обучения

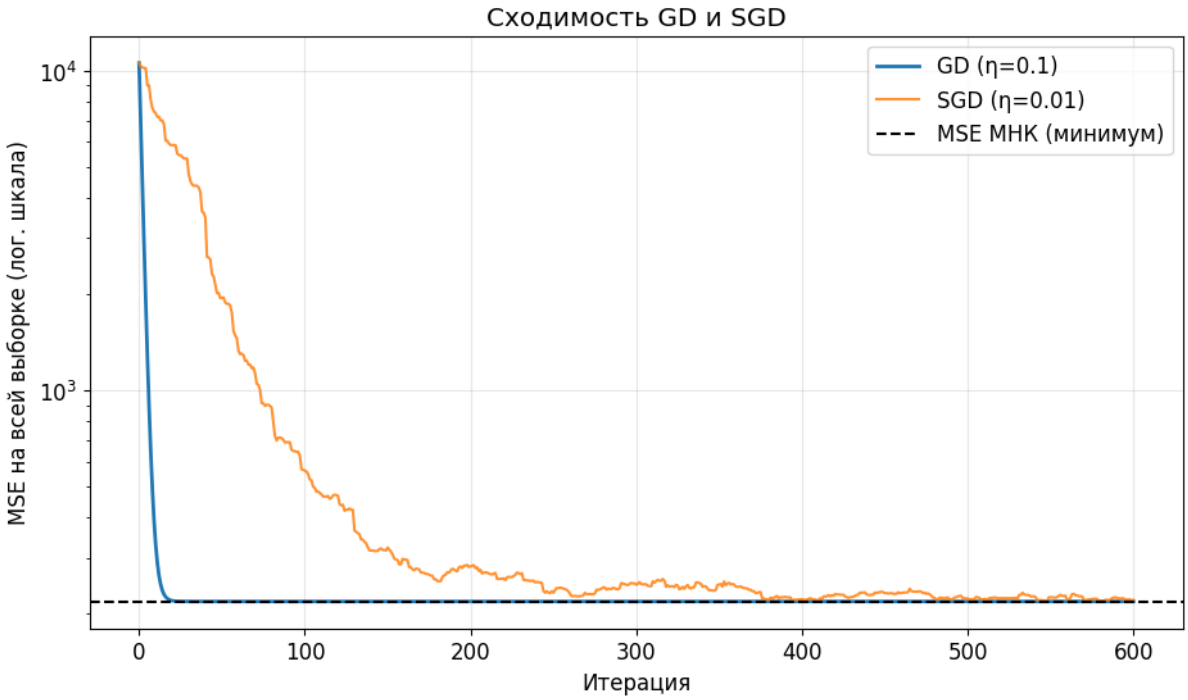


Рисунок 9 – Сравнение сходимости GD и SGD по числу итераций

```
Веса МНК: [ 8.584 24.983 82.875 51.637]
Веса GD : [ 8.584 24.983 82.875 51.637] | MSE = 218.898
Веса SGD: [ 9.522 25.443 81.752 52.023] | MSE = 221.415
```

Рисунок 10 – Веса и MSE, полученные методами МНК, GD и SGD

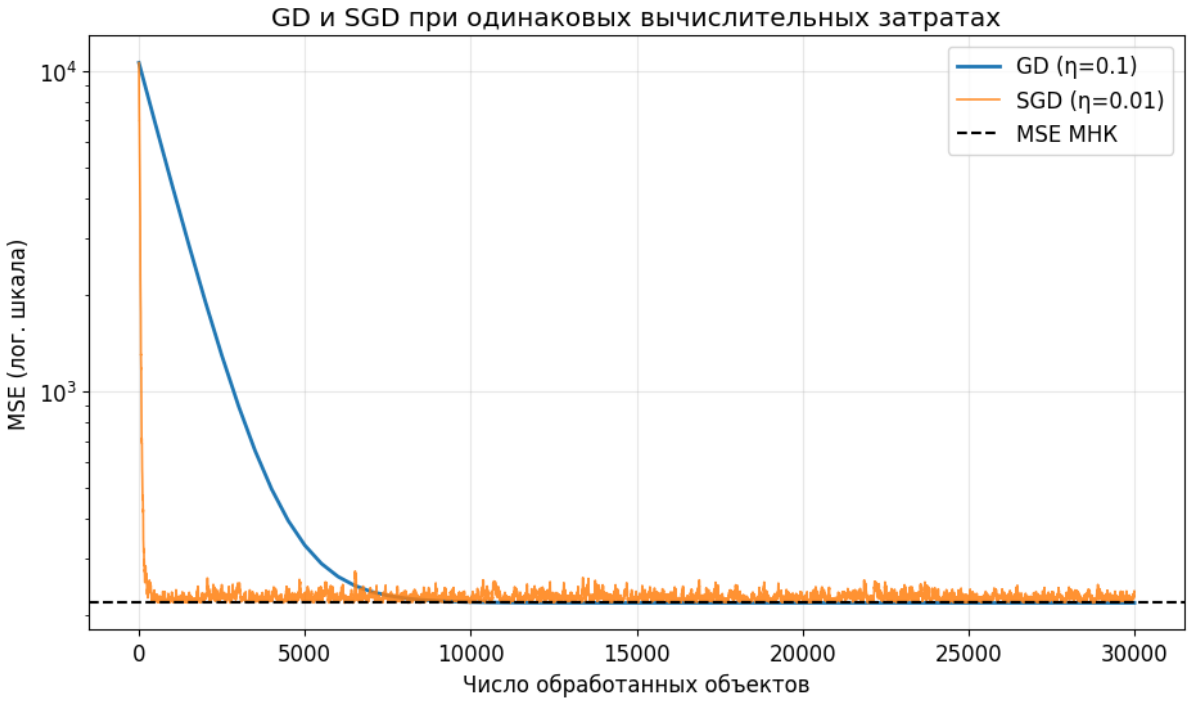


Рисунок 11 – Сравнение GD и SGD при одинаковых вычислительных затратах

```
MSE ≤ 229.8 (в пределах 5% от минимума):
  GD : 16 итераций = 8000 обработанных объектов
  SGD: 261 итераций = 261 обработанных объектов
```

Рисунок 12 – Число обработанных объектов до достижения MSE в пределах 5 % от минимума

```
Макс. отклонение GD от аналитического решения Ridge: 5.550143100663263e-08
Макс. число итераций: 10477
```

Рисунок 13 – Проверка градиентного спуска с L2-регуляризацией по аналитическому решению Ridge

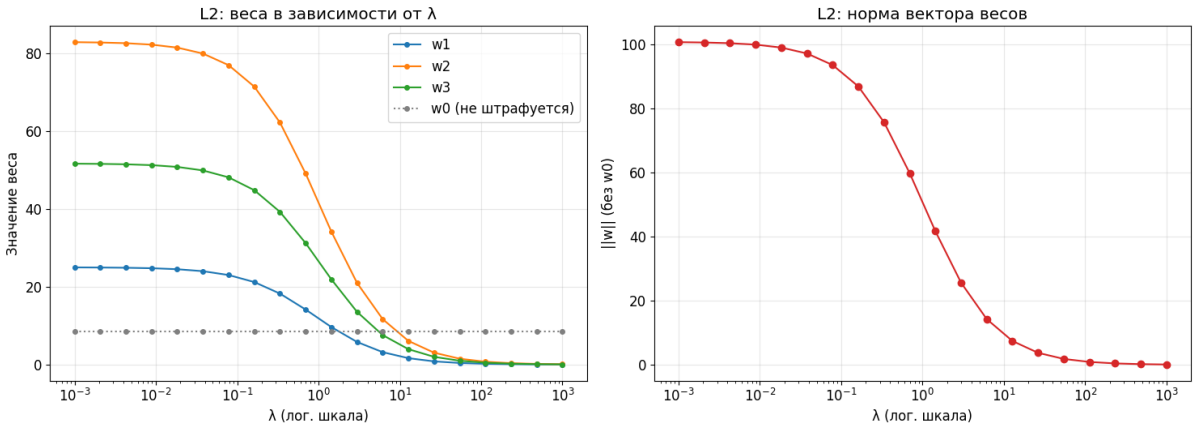


Рисунок 14 – Изменение весов и нормы весов в зависимости от коэффициента L2-регуляризации

λ	w0	w1	w2	w3	MSE
0.0010	8.5837	24.9553	82.7933	51.5885	218.9076
0.0021	8.5837	24.9260	82.7055	51.5367	218.9403
0.0043	8.5837	24.8656	82.5246	51.4296	219.0797
0.0089	8.5837	24.7415	82.1527	51.2096	219.6705
0.0183	8.5837	24.4885	81.3939	50.7601	222.1460
0.0379	8.5837	23.9810	79.8674	49.8547	232.2911
0.0785	8.5837	22.9946	76.8843	48.0797	272.0562
0.1624	8.5837	21.1896	71.3698	44.7798	415.1483
0.3360	8.5837	18.2253	62.1495	39.2086	856.7849
0.6952	8.5837	14.1279	49.0450	31.1770	1,922.4402
1.4384	8.5837	9.6362	34.1516	21.8920	3,761.8911
2.9764	8.5837	5.8091	20.9759	13.5428	5,950.4156
6.1585	8.5837	3.1876	11.6652	7.5688	7,816.6271
12.7427	8.5837	1.6481	6.0807	3.9569	9,063.5591
26.3665	8.5837	0.8243	3.0548	1.9910	9,779.2804
54.5559	8.5837	0.4052	1.5051	0.9817	10,156.7608
112.8838	8.5837	0.1975	0.7343	0.4792	10,347.2673
233.5721	8.5837	0.0958	0.3565	0.2327	10,441.3095
483.2930	8.5837	0.0464	0.1727	0.1127	10,487.2302
1,000.0000	8.5837	0.0224	0.0835	0.0545	10,509.5345

Рисунок 15 – Значения весов и MSE для 20 коэффициентов L2-регуляризации

6 Выводы

В ходе выполнения лабораторной работы были реализованы с нуля пакетный и стохастический градиентный спуск для линейной регрессии, исследовано влияние скорости обучения, проведено сравнение скорости сходимости методов и изучено влияние регуляризации на веса модели. На основании полученных результатов сделаны следующие выводы:

1) Сгенерирован датасет из 500 объектов с тремя информативными признаками (4 веса с учётом свободного члена); после стандартизации признаков градиентный спуск при $\eta = 0{,}1$ за 100 итераций вышел на веса [8,584; 24,983; 82,875; 51,637] и MSE = 218,898, полностью совпавшие с аналитическим решением МНК. Это подтверждает корректность реализации.
2) Скорость обучения определяет и скорость, и устойчивость сходимости. Для стандартизованных данных собственные числа гессиана лежат в диапазоне 1,89–2,09, поэтому порог устойчивости $\eta < 2/\lambda_{max} \approx 0{,}958$. При $\eta = 0{,}001$ за 300 итераций ошибка снизилась лишь до 3285,6 (минимум 218,9); при $\eta = 0{,}01$ для остановки по ε требуется около 1000 итераций, при $\eta = 0{,}1$ – 100; при $\eta = 0{,}5$ ($\approx 1/\lambda$) метод сошёлся за 9 итераций, а при $\eta = 0{,}9$, близкой к порогу, – за 183 итерации из-за колебаний; при $\eta = 1{,}0$ и выше метод расходится (MSE достигает 10^{25} и далее переполняется).
3) Масштабирование признаков существенно упрощает выбор η: после стандартизации собственные числа гессиана практически равны, одна скорость обучения подходит для всех весов, а порог устойчивости можно рассчитать заранее.
4) Стохастический градиентный спуск получен добавлением в ту же функцию выбора случайного объекта на каждой итерации. Его кривая ошибки немонотонная, «зубчатая». Чем больше η, тем выше уровень шума вокруг минимума: средняя MSE за последние 500 итераций составила 219,4 при $\eta = 0{,}001$ и 412,1 при $\eta = 0{,}1$; для сравнения выбрано компромиссное значение $\eta = 0{,}01$.
5) По числу итераций GD сходится быстрее и точнее: ошибка достигает минимума (218,898) примерно за 20 итераций, тогда как SGD за 600 итераций доходит до 221,4 с весами [9,52; 25,44; 81,75; 52,02] и продолжает колебаться около минимума. Однако по реальным вычислительным затратам выигрывает SGD: уровня «минимум + 5 %» он достигает, обработав 261 объект, тогда как GD – 8000 объектов (16 итераций по 500 объектов), то есть примерно в 30 раз больше. На

больших выборках SGD (или мини-батч) практичнее, а для точной сходимости необходимо уменьшать η по ходу обучения.

6) Реализована L2-регуляризация (вариант 7) градиентным спуском со свободным членом без штрафа и шагом $\eta = 0{,}9/(1 + \lambda)$. Результат для 20 значений λ от 0,001 до 1000 совпал с аналитическим решением Ridge (максимальное расхождение $5{,}6 \cdot 10^{-8}$). При $\lambda = 0{,}001$ веса почти равны весам МНК (24,96; 82,79; 51,59), а при $\lambda = 1000$ они стремятся к нулю (0,022; 0,084; 0,055), при этом точного нуля не достигают; свободный член остаётся равным среднему значению y (8,584). MSE при этом растёт с 218,9 до 10509,5, то есть приближается к дисперсии целевой переменной – модель недообучена.